

Music Playlist Manager

Progettazione Architeturale

Progetto Music Playlist Manager
Corso Software Architecture Design
Gruppo Gruppo 04

Indice

1	Progettazione Architeturale	2
1.1	<u>Vista Logica</u>	2
1.1.1	Model	2
1.1.2	View	2
1.1.3	Controller	4
1.2	<u>Vista di Processo</u>	5
1.3	Scenari	5
1.3.1	<u>UC — Gestione Tracce</u>	6
1.3.2	<u>UC — Gestione Playlist</u>	6
1.3.3	<u>UC — Riproduzione Musica</u>	7
1.3.4	<u>UC — Modalità Riproduzione Playlist</u>	7
2	<u>Architettura dopo lo Sprint 1</u>	8
2.1	Pattern introdotti / consolidati	8
2.2	Stato di avanzamento	8
2.3	Mappatura: componenti logici → classi implementate	9
2.4	Diagrammi delle classi	10
2.4.1	Pattern Observer	13
2.5	Novità rispetto al design iniziale	14

Progettazione Architetturale

La descrizione dell'architettura del sistema adotta parzialmente il modello **4+1 View Model** di Kruchten, focalizzandosi principalmente sulla **Vista Logica** (struttura statica e responsabilità) e sulla **Vista di Processo** (dinamica a runtime e concorrenza).

Vista Logica

Per garantire alta coesione e basso accoppiamento, è stato adottato il pattern architetturale **Model-View-Controller (MVC)**.

Di seguito vengono dettagliate le responsabilità dei singoli componenti:

Model

Il livello *Model* è composto da quattro entità principali:

1. **Entità traccia audio** — Rappresenta la singola traccia con i relativi metadati: titolo, autore, durata, genere, anno di pubblicazione.
2. **Entità collezione di tracce** — Rappresenta una collezione ordinata e nominata di tracce. Supporta operazioni di aggiunta, rimozione e riordinamento degli elementi.
3. **Entità stato di riproduzione** — Gestisce lo stato della riproduzione corrente (IN PAUSA / IN RIPRODUZIONE / FERMO) e la strategia di avanzamento tra le tracce.
4. **Entità catalogo centrale** — Aggrega tutte le tracce e le playlist disponibili nel sistema, fungendo da punto di accesso unico alla libreria musicale.

View

Il livello *View* è articolato in sei componenti visive:

1. **Vista di inserimento traccia** — Dedicata all'aggiunta di una nuova traccia, comprende i campi per i metadati, il caricamento del file audio e l'immagine associata.
2. **Vista elenco tracce** — Visualizza l'elenco completo delle tracce registrate nel sistema.
3. **Vista elenco playlist** — Visualizza le playlist create dall'utente.
4. **Vista dettaglio playlist** — Visualizza le tracce contenute in una specifica playlist selezionata.
5. **Finestra principale** — Funge da contenitore per le viste relative alle tracce e alle playlist.
6. **Barra di riproduzione** — Mostra le informazioni sulla traccia in corso (immagine, titolo) e i controlli di interazione:
 - a. Bottoni di:
 - i. Play/Stop

- ii. Skip traccia
- b. Immagine della traccia in riproduzione, con Titolo della Traccia Audio.

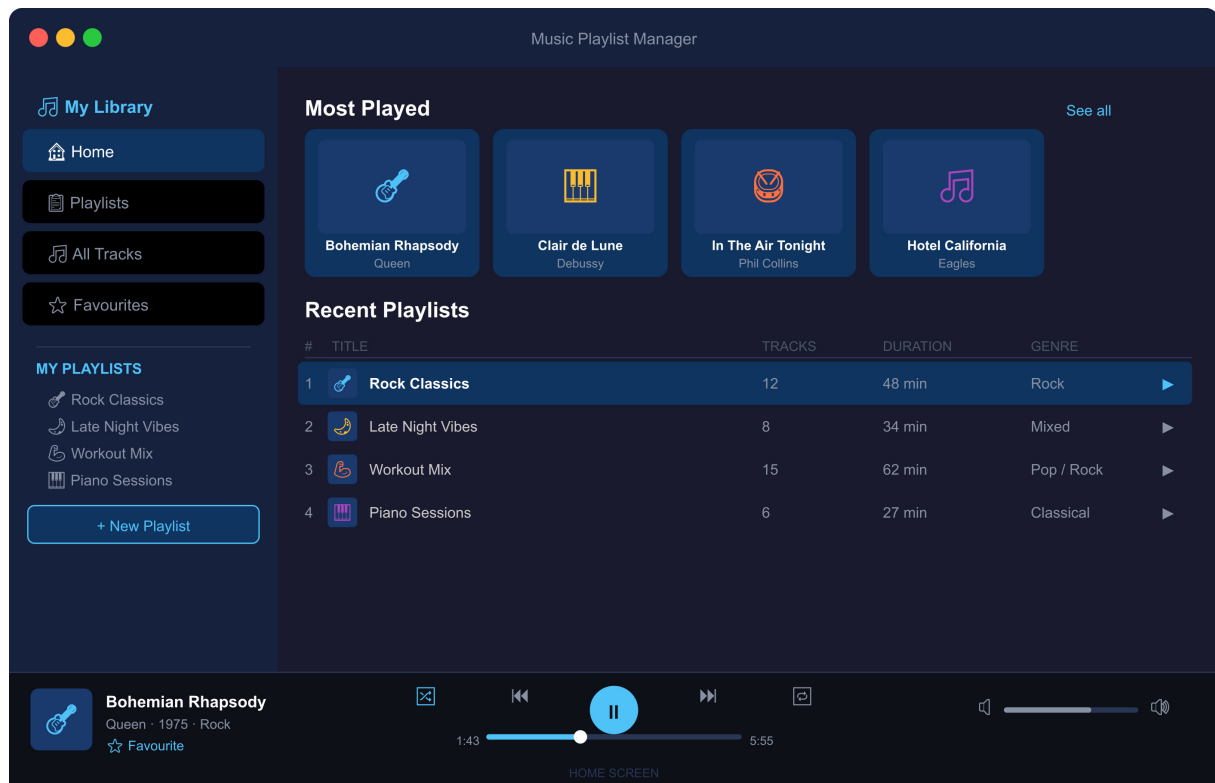


Figura 1: Mockup Home

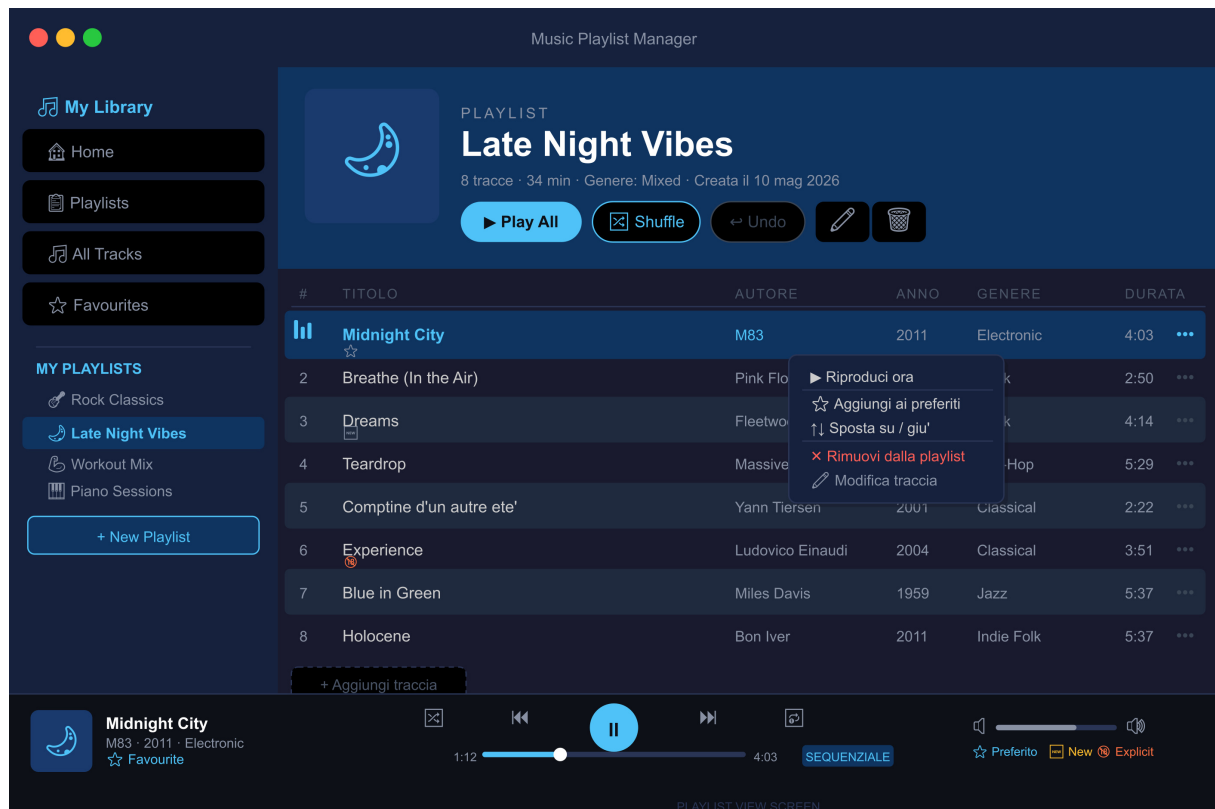


Figura 2: Mockup Dettaglio Playlist

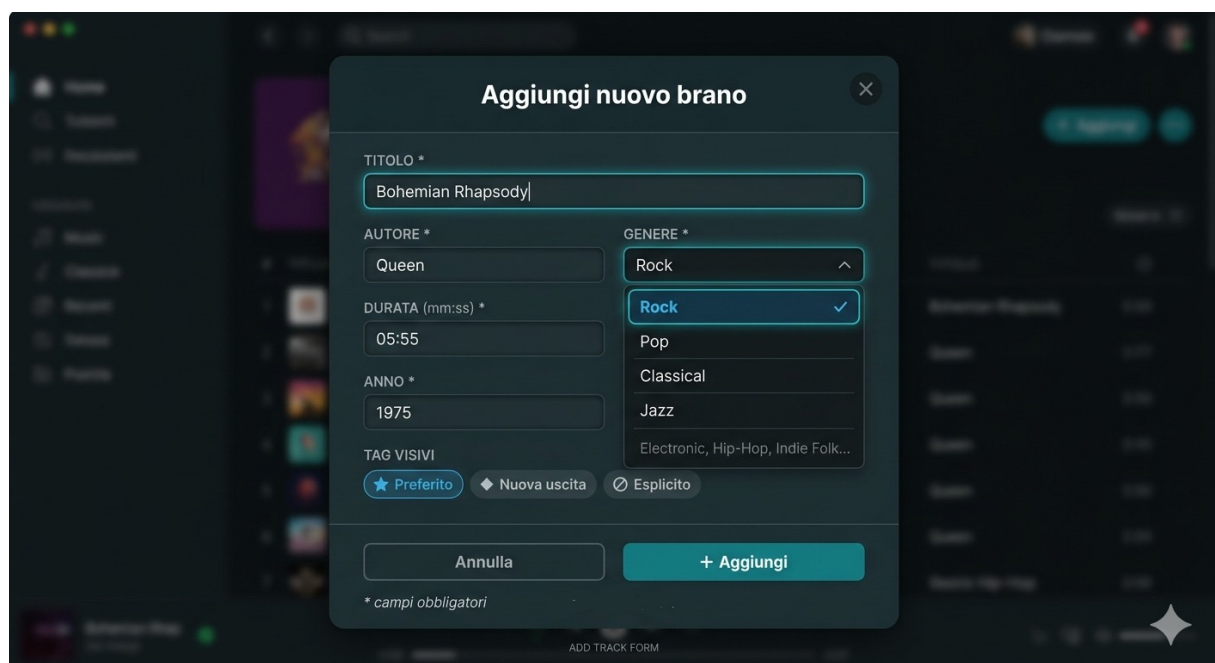


Figura 3: Mockup Form di Inserimento Brani

Controller

Il livello *Controller* è composto da tre componenti:

1. **Controller delle playlist** — Responsabile della gestione delle playlist, con funzionalità di creazione, rinomina e rimozione.
2. **Controller delle tracce** — Responsabile della gestione del catalogo tracce, con funzionalità di aggiunta, rimozione e modifica.
3. **Controller di riproduzione** — Responsabile della gestione della riproduzione, con funzionalità di play, pausa e salto alla traccia successiva.

Vista di Processo

Nel sistema **Music Playlist Manager** l'unica problematica di concorrenza significativa riguarda la gestione della riproduzione musicale simulata.

Durante la riproduzione di una traccia, il sistema deve aggiornare costantemente la barra di avanzamento garantendo al contempo la piena reattività dell'interfaccia grafica, in modo che l'utente possa continuare ad effettuare operazioni senza interruzioni.

In particolare, il sistema deve assicurare che, durante la riproduzione:

- la barra di avanzamento si aggiorni in tempo reale;
- l'utente possa interagire liberamente con l'interfaccia (navigazione tra schermate, gestione di tracce e playlist);
- la traccia successiva venga caricata automaticamente al termine del brano corrente o al cambio della playlist.

Questa soluzione consente di mantenere l'applicazione reattiva, garantendo un aggiornamento continuo e fluido della riproduzione musicale.

Scenari

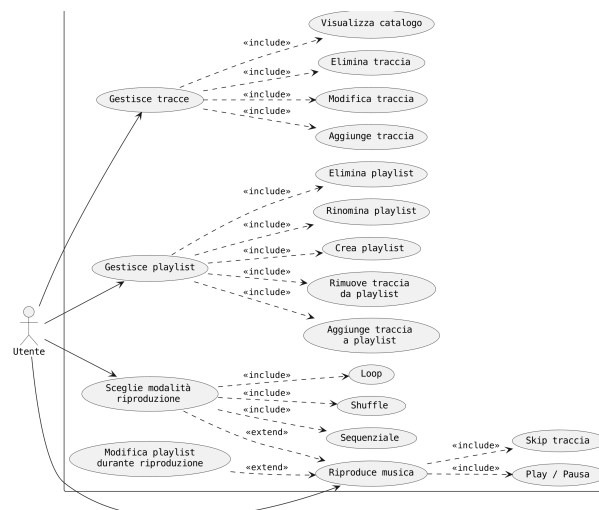


Figura 4: Vista di Scenario

UC — Gestione Tracce

Scenario A: Inserimento di una nuova traccia

1. L'utente seleziona l'opzione per aggiungere una nuova traccia.
2. L'utente inserisce i dati della canzone (titolo, autore, durata, genere, anno) e preme il pulsante di conferma.
3. Il sistema valida i dati e inserisce la nuova canzone nel catalogo musicale.

Scenario B: Modifica di una traccia esistente

1. L'utente seleziona una traccia dal catalogo e preme il pulsante di modifica.
2. L'utente modifica uno o più campi e preme il pulsante di salvataggio.
3. Il sistema aggiorna i dati della traccia all'interno del catalogo.

Scenario C: Eliminazione di una traccia

1. L'utente seleziona una traccia dal catalogo e preme il pulsante "Elimina".
2. Il sistema rimuove in modo permanente la canzone dal catalogo generale.
3. Il sistema aggiorna il catalogo rimuovendo visivamente il brano in tempo reale.

Scenario D: Visualizzazione Catalogo

1. L'utente seleziona il catalogo.
2. Il sistema mostra all'utente tutte le tracce contenute nel catalogo.

UC — Gestione Playlist

Scenario A: Inserimento di una nuova Playlist

1. L'utente seleziona l'opzione per aggiungere una nuova playlist.
2. L'utente inserisce il nome della playlist.
3. Il sistema valida i dati e inserisce la nuova playlist nella lista.

Scenario B: Modifica di una Playlist esistente

1. L'utente vuole rimuovere o inserire una traccia nella playlist.
2. Il sistema aggiorna i brani della playlist.

Scenario C: Eliminazione di una Playlist

1. L'utente seleziona una playlist e preme il pulsante "Elimina".
2. Il sistema rimuove in modo permanente la playlist.
3. Il sistema aggiorna la lista rimuovendo visivamente la playlist in tempo reale.

Scenario D: Rinomina Playlist

1. L'utente seleziona la modifica nome della playlist.
2. L'utente modifica il nome della playlist.
3. Il sistema mostra all'utente la playlist con il nome aggiornato.

UC — Riproduzione Musica

1. L'utente seleziona una traccia o una playlist.
2. L'utente preme Play.
3. Il sistema avvia la riproduzione dalla prima traccia disponibile.
4. Il sistema aggiorna la barra di riproduzione con titolo e stato IN RIPRODUZIONE.
5. Il sistema avanza automaticamente alla traccia successiva secondo la modalità attiva.

UC — Modalità Riproduzione Playlist

1. L'utente seleziona una playlist.
2. L'utente preme l'apposito pulsante per fare switching della modalità di riproduzione.
3. Il sistema cambia la strategia di riproduzione tramite il componente di stato condiviso.
4. Il sistema aggiorna la barra di riproduzione con titolo e stato IN RIPRODUZIONE della prima traccia a seconda della modalità scelta.
5. Il sistema avanza automaticamente alla traccia successiva secondo la modalità attiva.

Architettura dopo lo Sprint 1

Rispetto alla progettazione iniziale (Pre-Game), durante lo Sprint 1 l'architettura è stata raffinata in fase di implementazione. Le scelte di fondo (MVC, modello 4+1) restano valide; di seguito le evoluzioni e la relativa motivazione.

Pattern introdotti / consolidati

1. **Observer** — Il catalogo centrale è stato realizzato come *Subject*: a fronte di modifiche al Model notifica le viste registrate (*Observer*) tramite eventi tipizzati. Disaccoppia le mutazioni del Model dall'aggiornamento della View, che si limita a re-renderizzarsi alla ricezione dell'evento.
2. **Programmazione verso le interfacce** — Le entità del Model (traccia, collezione, catalogo) sono separate in interfaccia + implementazione, riducendo l'accoppiamento (dependency inversion).

Raffinamenti del Model

- Il catalogo funge da *aggregate root* e da *Subject* unico.
- Le collezioni esposte dalle entità sono restituite in forma **immutabile**, a tutela dell'incapsulamento.

Stato di avanzamento

- **Implementato nello Sprint 1:** gestione tracce e playlist (CRUD), catalogo, aggiornamento reattivo della UI via Observer.
- **Differito allo Sprint 2:** barra e logica di riproduzione, strategie delle modalità di ascolto.

Mappatura: componenti logici → classi implementate

Livello	Classe / Interfaccia	Ruolo
Model	Track, TrackImpl	Traccia audio e relativi metadati
	Playlist, PlaylistImpl	Collezione ordinata e nominata di tracce
	MusicCatalog,	Catalogo centrale (<i>aggregate root</i> ; anche <i>Subject</i>
	ConcreteMusicCatalog	del meccanismo Observer)
Observer	CatalogObserver	Interfaccia osservatore del catalogo
	CatalogEvent	Evento di modifica del catalogo
	CatalogEventType	Enum dei tipi di evento (PLAYLIST_ADDED, ...)
Controller	PlaylistController	Creazione, rinomina, rimozione delle playlist
	TrackController	Aggiunta, rimozione, modifica delle tracce
View	MainViewController	Finestra principale / contenitore
	TrackListViewController	Elenco completo delle tracce (catalogo)
	PlaylistViewController	Elenco delle playlist
	PlaylistDetailViewController	Tracce di una playlist selezionata
	TrackFormViewController	Inserimento / modifica di una traccia
	TrackSelectionViewController,	Selezione di una traccia da aggiungere a una playlist
	TrackSelectionListener	
Util	TrackFormatter	Formattazione dei dati di una traccia (es. durata MM:SS)
	TableColumnFactory	Configurazione delle colonne delle TableView che mostrano tracce musicali

Diagrammi delle classi

La struttura statica del sistema è documentata tramite due diagrammi delle classi, suddivisi per area di responsabilità per garantirne la leggibilità: il primo raccoglie il *Model* e il meccanismo di notifica (*Observer*), il secondo i *Controller* e le *View*. La suddivisione rispecchia l'organizzazione in package del progetto ed evita un unico diagramma poco leggibile.

Come mostrato in Figura 5, ogni entità del Model è definita da un'**interfaccia** e dalla relativa **implementazione** (*Track/TrackImpl*, *Playlist/PlaylistImpl*, *MusicCatalog/ConcreteMusicCatalog*), a vantaggio del basso accoppiamento e della sostituibilità. *ConcreteMusicCatalog* ricopre il ruolo di *Subject* del pattern *Observer*: mantiene la lista degli osservatori (*CatalogObserver*), genera eventi tipizzati (*CatalogEvent*, classificati da *CatalogEventType*) e li notifica a ogni modifica del catalogo. Gli osservatori concreti sono i controller di vista, riportati nel secondo diagramma.

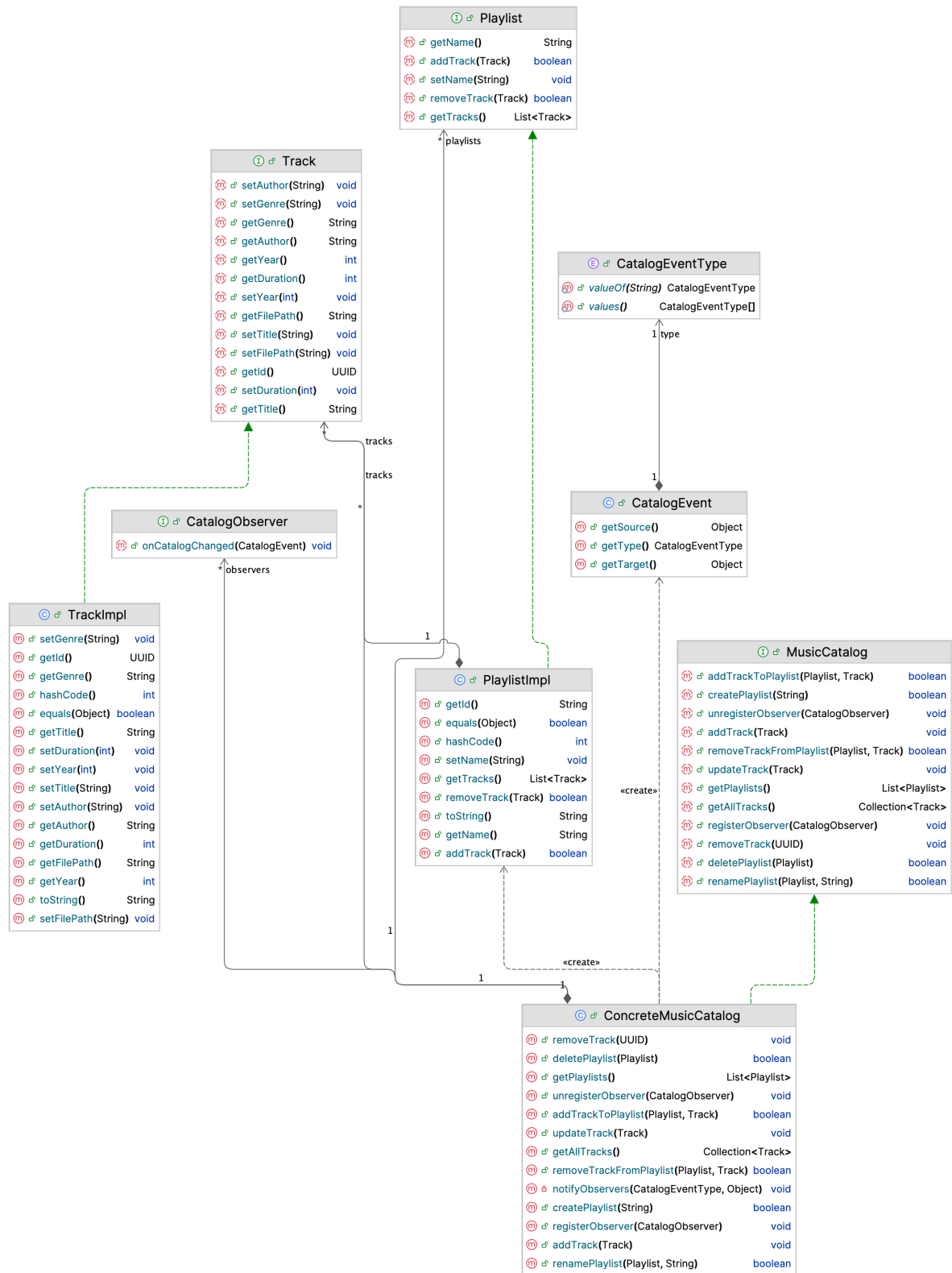


Figura 5: Diagramma delle classi — Model e Observer

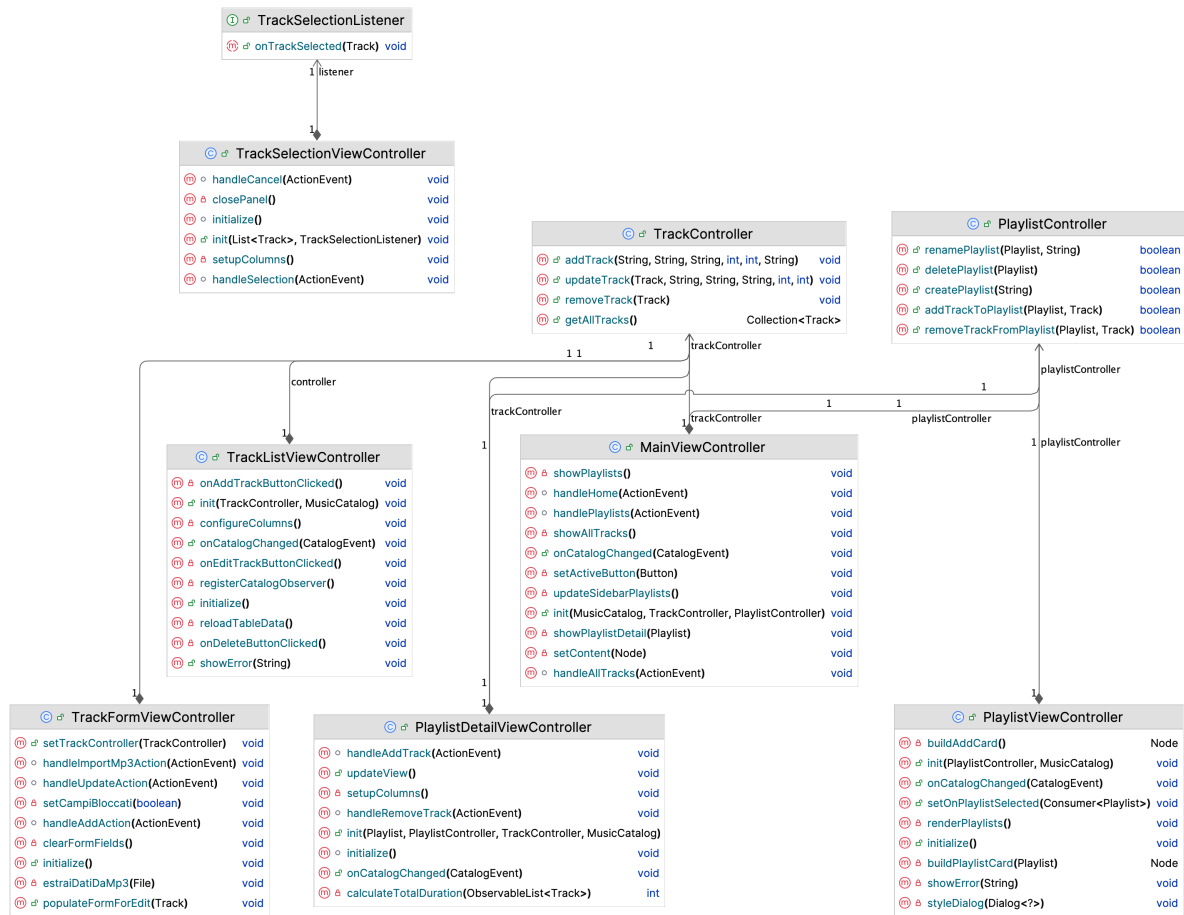


Figura 6: Diagramma delle classi — Controller e View (MVC)

La Figura 6 mostra i Controller e le View. I controller (`PlaylistController`, `TrackController`) espongono le operazioni di dominio e le delegano al catalogo. I controller di vista implementano `CatalogObserver`: ricevuto un evento, aggiornano l'interfaccia, realizzando il disaccoppiamento tra la modifica del Model e l'aggiornamento della View. Completano il quadro `TrackSelectionListener` (callback per la selezione di una traccia da aggiungere a una playlist) e `TrackFormatter` (utility di formattazione dei metadati).

Pattern Observer

L'aggiornamento automatico dell'interfaccia è realizzato applicando il pattern **Observer**. La Figura 7 ne mostra i ruoli mappati sui componenti del sistema; la corrispondenza con i ruoli canonici del Gang of Four è riportata in Tabella 1.

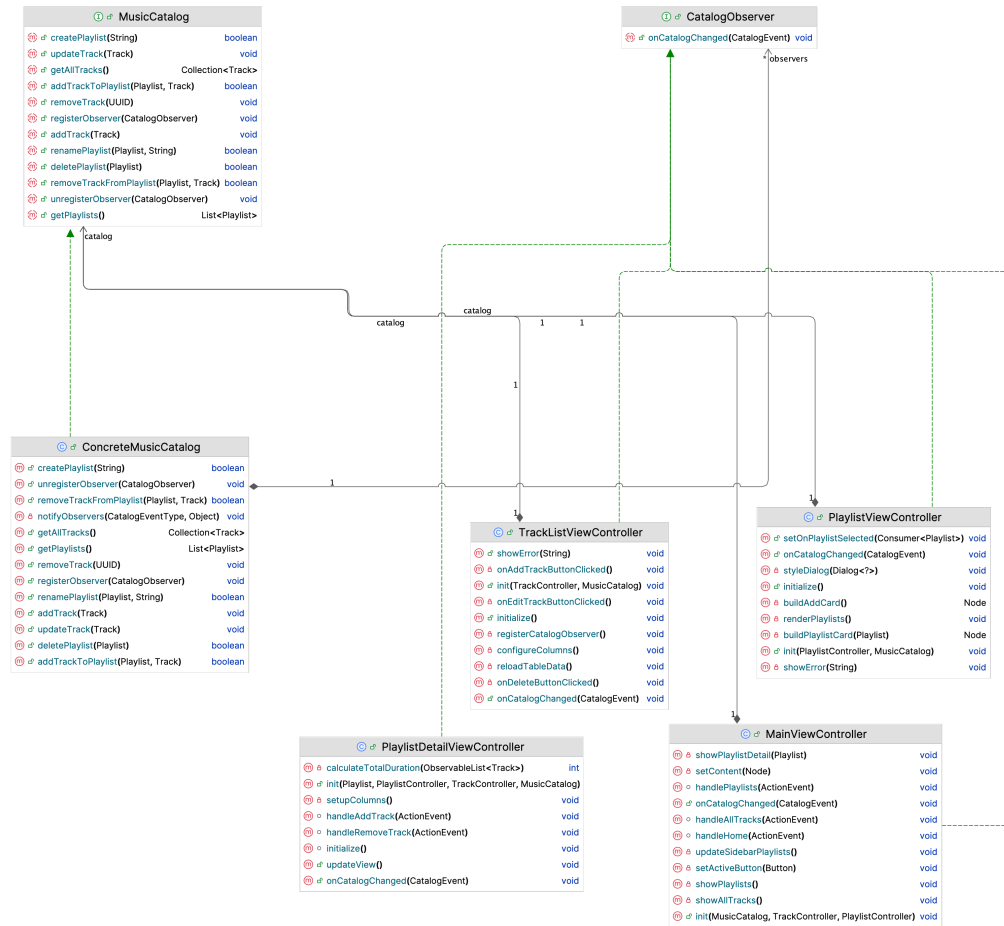


Figura 7: Pattern Observer applicato al sistema

Ruolo	Componente nel sistema
Subject	MusicCatalog (interfaccia: <code>registerObserver</code> / <code>unregisterObserver</code>)
ConcreteSubject	ConcreteMusicCatalog (mantiene la lista <code>observers</code> e implementa <code>notifyObservers</code>)
Observer	CatalogObserver (<code>onCatalogChanged</code>)
ConcreteObserver	I view controller: <code>TrackListViewController</code> , <code>PlaylistViewController</code> , <code>PlaylistDetailViewController</code> , <code>MainViewController</code>

Tabella 1: Mappatura dei ruoli del pattern Observer

A ogni modifica del catalogo (aggiunta, rimozione o modifica di tracce o playlist), `ConcreteMusicCatalog` invoca `notifyObservers`, che scorre la lista degli osservatori registrati e chiama `onCatalogChanged` su ciascuno; il view controller, ricevuta la notifica,

rilegge lo stato dal catalogo e ri-renderizza l'interfaccia. In questo modo le mutazioni del Model sono disaccoppiate dall'aggiornamento della View.

Rispetto allo schema canonico si segnalano due scelte implementative:

- Il ruolo *Subject* è un'interfaccia (`MusicCatalog`) e non una classe astratta. La lista degli osservatori e il metodo di notifica risiedono nella classe concreta `ConcreteMusicCatalog`.
- La notifica adotta il modello **event-based**: `onCatalogChanged` riceve un oggetto-evento `CatalogEvent` che trasporta sorgente, tipo e target della modifica, mentre `CatalogEventType` ne classifica la tipologia (`PLAYLIST_ADDED`, `TRACK_REMOVED`, ...) permettendo agli osservatori di leggere le notifiche di interesse.

Novità rispetto al design iniziale

- **Pacchetto observer**: introdotto per realizzare il pattern Observer (assente nel Pre-Game), con eventi tipizzati.
- **Separazione interfaccia/implementazione** nel Model (`Track/TrackImpl`, `Playlist/PlaylistImpl`, `MusicCatalog/ConcreteMusicCatalog`).
- **Vista di selezione traccia** (`TrackSelectionViewController`), non prevista inizialmente, per l'aggiunta di brani ad una playlist.
- **Interfaccia `TrackSelectionListener`**, per la comunicazione tra `TrackSelectionViewController` e il chiamante. È un'interfaccia funzionale con un unico metodo `onTrackSelected(Track)`, che permette di passare una lambda come callback all'apertura del pannello. Questo disaccoppia la vista dal contesto in cui viene utilizzata, rendendola riusabile senza modifiche.
- **Utility `TableColumnFactory`** per la configurazione delle colonne delle `TableView` contenenti tracce musicali. L'introduzione di questa classe è motivata dal principio *Don't Repeat Yourself (DRY)*: la logica di configurazione delle colonne era duplicata in tutte le view che visualizzano tracce (`PlaylistDetailViewController`, `TrackListViewController`, `TrackSelectionViewController`). Centralizzando tale logica in un unico metodo statico, si ottiene inoltre un *Single Point of Change*: qualora si aggiungesse o modificasse un campo della classe `Track`, è sufficiente intervenire esclusivamente su `TableColumnFactory` senza dover propagare la modifica a tutte le view coinvolte.
- **Utility `TrackFormatter`** per la formattazione dei metadati.